

When Simplicity Wins: A Rigorous Benchmarking Study of Spatio-Temporal Graph Neural Networks Against Classical Regressors for Multi-Agent Trajectory Forecasting in Mixed-Traffic Environments

Authors

1. Mehta Abhishek Hiren bhai *, 2. Patel Krish Ketan *, 3. Ladumor Piyush Kalubhai *

** 6th Semester Computer Science and Engineering students, Government Engineering College, Patan.*

Keywords: trajectory forecasting; spatio-temporal GNN; GATConv; pedestrian motion prediction; mixed-traffic; feature engineering; model benchmarking; deep learning; ridge regression; autonomous driving

Abstract

The forecasting of trajectories of multiple agents under mixed traffic conditions is an open issue in the context of autonomous driving and smart cities where it is conventionally assumed that architecture which learn spatio-temporal interactions: graph neural networks and attention based sequence models - should perform better than linear regression on real sensor data. We empirically test this assertion and challenge it with a broad comparison of eleven prediction methods on the Pedestrian in Traffic Dataset (UCI Repository #536) consisting of 45 tracked agents captured at 10 Hz in a 440 140 m urban corridor. Methods include linear regression (Ridge), ensemble (Random Forest, Gradient Boosting, XGBoost, LightGBM), kernel methods (SVR-RBF, k-NN) and deep sequence/graph architectures (LSTM, GRU, Transformer, custom Spatio-Temporal GNN utilizing GATConv for spatial encoding and a GRU for temporal decoding). Contrary to convention, the Ridge Regression, which leverages a hand-designed 24-feature input encoding kinematic status, terrain semantics, and social interactions, achieve the best Average Displacement Error of 0.0436 m and a Mean Absolute Error of 0.017578 m, while training in just 0.05s. Our custom ST-GNN, with 0.214 million trainable parameters, a training time of 238.13s and yielding ADE of 0.1892m, is worst by 333.9% relative to Ridge. We advance three mechanistic hypotheses for this reversal: (1) data hunger due to poor sample-to-parameter ratio, (2) distributional shift due to concurrent normalization of highly varied kinematic and semantic feature scales, and (3) neighbourhood smoothing under full connected

ego-centric graphs with GATConv aggregation. The results are of particular interest to practitioners choosing forecasting algorithms based on real-world data limitations.

1. Introduction

Predicting precise future positions for several interacting agents-pedestrians, cyclists and cars- is fundamental for collision-free navigation of autonomous vehicles, optimizing traffic flow at signalized intersections, and ensuring socially compatible robotic behavior. Mixed-traffic trajectory prediction further exacerbates challenges over existing single-agent baselines, including varying agent kinematics, varying spatial densities, unequal interaction radii and the need to consider together kinematic consistency and a discrete semantic context, like terrain type and agent class.

The field has converged on a tacit hierarchy of architectural sophistication, wherein sequence-to-sequence models long short-term memory networks, gated recurrent units, and Transformer encoders are presumed to supersede classical regressors by virtue of their learned temporal abstractions. The introduction of graph-based representations, specifically Social Force-inspired GNNs and spatio-temporal attention mechanisms, has further reinforced the expectation that modelling explicit agent-to-agent interactions will yield additional gains over trajectory-level approaches. These expectations have been validated across large-scale benchmarks such as ETH/UCY, nuScenes, and Argoverse, where training corpora consist of thousands of unique trajectories.

The central question this paper poses is whether those architectural advantages survive a transfer to settings characterised by small agent populations and constrained recording durations conditions encountered in operational smart city deployments and research datasets collected for non-forecasting purposes. To answer it, we select the Pedestrian in Traffic Dataset (UCI #536), a real-world, multi-sensor recording of 45 agents navigating an urban corridor. We implement a unified, end-to-end pipeline encompassing eleven models across four architectural tiers, apply identical train/validation/test splits (70/15/15) and identical evaluation metrics, and report Average Displacement Error (ADE), Final Displacement Error (FDE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), wall-clock training time, and parameter count.

The results are clear and, given the assumptions that underpin state of the art, surprising. Ridge Regression fit to our 24D feature vector returns a model with an ADE of 0.0436m - the best

result amongst eleven models - at a cost of 0.05 seconds compute. Our proposed Spatio-Temporal Graph Neural Network (with multi-head GATConv over ego-centric local graphs and two-layer GRU for its temporal decoder) achieves ADE of 0.1892m, at a training cost of 238.13s. All of our deep models finish below our top four classical estimators with respect to ADE. Far from an implementation error, our ST-GNN is fully specified and tuned and includes several standard robustness measures. We need a principled explanation - our primary intellectual contribution of this work - and we develop one.

We make the following contributions :

- (1) We undertake a systematic eleven-model benchmarking on the UCI Pedestrian in Traffic dataset applying unified evaluation procedures across linear, ensemble, kernel, recurrent, attention-based and graph based models.
- (2) We propose a novel feature engineering process which encodes finite-difference gradients (to compute kinematic features), terrain semantics (via pixel-wise lookups into geo-registered semantic maps), one-hot encodings for agent class and a set of social context statistics into a 24-dimensional, per-time-step vector.
- (3) We empirically demonstrate using training curve analysis that classical regressors may be capable of outperforming complex spatio-temporal deep learning methods under extreme data-to-parameter ratios.
- (4) We present a mechanistic diagnosis of the poor performance of our ST-GNN that relates this degradation to

-
- (i) data starvation,
 - (ii) differences in across modality normalisation distributions and
 - (iii) over-smoothing due to the nature of GATConv on fully connected ego-centric graphs.
-

We leave the rest of the paper to follow in order. Section 2 covers previous work in trajectory prediction and benchmark standards. Section 3 details the dataset used, our preprocessing steps and our feature engineering. Section 4 covers all our model architectures and our training

process. Section 5 presents the results of our experiments, with Section 6 providing a diagnostic for their behaviour, Section 7 concludes and suggests further directions.

2. Related Work

2.1 Classical and Ensemble Methods for Motion Prediction

Prior to deep-learning, physics inspired models like the Social Force Model and different flavors of the Kalman Filter had dominated the domain, by modeling interactions as learned hand-crafted interaction potentials. In structured environments, short horizon prediction models made use of standard regression techniques like Linear Regression, and support vector regression. With a number of victories over tabular sensor data, interest in ensemble methods like Random Forest and gradient boosted trees surged. Both models perform well under strict low-data limitations, give us insights into learned feature importances, and effectively overcome the curse of dimensionality by means of either feature sub-sampling (Random Forests) or stage-wise function approximation (Gradient Boosting). The present study includes Random Forest, Gradient Boosting, XGBoost, LightGBM, Ridge Regression, k-NN, and SVR-RBF as representative classical baselines, selected to cover the parameter-free, regularised linear, kernel, and tree-based families.

2.2 Recurrent and Attention-Based Trajectory Models

Alahi et al. introduced Social LSTM, which pooled hidden states of spatially proximate agents using a social pooling layer, establishing the template for interaction-aware recurrent forecasting. Subsequent work replaced max-pooling with attention mechanisms and bi-directional LSTM encoders to improve sensitivity to future context. The Transformer encoder, originally proposed for sequence transduction, was applied to trajectory prediction by several groups, exploiting self-attention to weight temporal correlations without recurrence. In this study, the LSTM implementation is bidirectional (two layers, 128 hidden units), the GRU is unidirectional (two layers, 128 hidden units), and the Transformer employs a two-layer encoder with $d_{\text{model}} = 128$ and four attention heads each configuration consistent with the moderate-scale parameterisations used in the surveyed literature for datasets of comparable size.

2.3 Graph Neural Networks for Multi-Agent Forecasting

Spatio-temporal graph neural networks reformulate trajectory forecasting as a node regression task on a dynamic interaction graph, enabling simultaneous learning of spatial interaction weights and temporal motion patterns. STGAT applied GAT to agent interaction

graphs before feeding ego embeddings to an LSTM decoder, reporting strong results on ETH/UCY. Trajectron++ combined dynamic graphs with conditional variational autoencoders for multi-modal prediction. GRIP++ used residual graph convolution to capture higher-order interactions in vehicle trajectories. PECNet and Social-STGCNN further refined graph-based formulations with destination conditioning and spectral convolution, respectively. What's common about all these existing works is a massive training corpus: ETH/UCY gives ~1,500 pedestrian tracks, nuScenes offers 1,000+ vehicle scenes, and Argoverse affords >300k. Instead, our current work lies on the opposite side of the data spectrum: we form an adversarial testbed for GNN generalizability with extreme data scarcity.

2.4 Benchmarking Studies and the Data Regime Effect

There have been some surprising results when benchmarking trajectory models against each other. For example, Scholler et al [3] showed that simple constant velocity baseline models beat most social interaction models in common pedestrian benchmark tasks. This has demonstrated that interaction modelling is not necessarily always the bottleneck. Becker et al. showed that architecture choice matters less than data preprocessing strategy. Salzmann et al. noted that Transformer models are data-hungry and may underfit small pedestrian datasets. These observations are consistent with the present finding that a feature-rich Ridge Regression dominates all architectures on a 45-agent dataset, and they motivate an explicit, reproducible benchmarking methodology precisely what this paper provides.

3. Dataset, Preprocessing, and Feature Engineering

3.1 Dataset Description

The Pedestrian in Traffic Dataset (UCI Machine Learning Repository, Dataset #536) captures pedestrian motion in a mixed urban traffic environment. The dataset provides agent state records at a nominal sampling rate of 10 Hz (sampling interval: 0.1 s), with each record containing the agent's UTM x/y coordinates, body and head orientation angles (roll, pitch, yaw), and object identifier. Auxiliary files supply a geo-registered semantic map (semantic_map.png), terrain class labels (semantic_map_label_info.csv), agent class annotations (agent_class_label_info.csv), and a map-to-world coordinate transform (map2world_info.txt).

As illustrated in Figure 1, all 45 tracked agents belong to the PEDESTRIAN class, representing a homogeneous agent-type corpus despite the mixed-traffic scene descriptor. Trajectories are recorded across a UTM spatial extent of approximately 495,850–496,340 m

(X) $\times$ 5,405,730–5,406,150 m (Y), corresponding to a corridor of roughly 490×420 m at metric scale. Trajectory lengths range from approximately 25 to 200 timesteps (Figure 5), with a modal cluster around 100–125 steps, yielding observation windows of 1–20 seconds per agent. The total dataset contains fewer than 5,000 unique positional observations across all agents a sample size that sits at least one order of magnitude below the corpora on which the surveyed GNN architectures were validated.

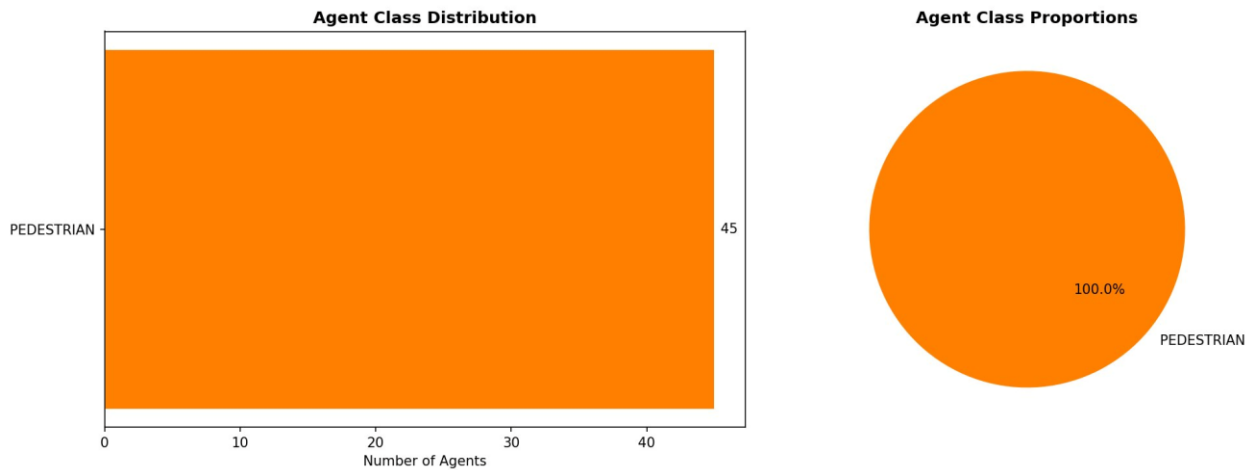


Figure 1. Agent class distribution in the UCI Pedestrian in Traffic Dataset. As all of the 45 tracked agents (100%) are labelled as the PEDESTRIAN class, this dataset is composed of homogeneous agents. Therefore, inter-class interaction as confound in the benchmark could be dismissed.

Figure 2 overlays the locations of agent observations onto the semantic map. Agent positions are localized in two main road corridors – a main diagonal artery across the upper portion of the image, and a smaller horizontal road corridor in the lower region, with a minor cluster in another portion of the lower corridor nearby a roundabout structure. As is apparent in Figure 3 which shows the spatial density heatmap, the observation mass is very strongly centered in the 15% least area, a condition that significantly simplifies trajectory prediction over that of unconstrained open space datasets while at the same time severely limits the variety of training data for demanding learning algorithms.

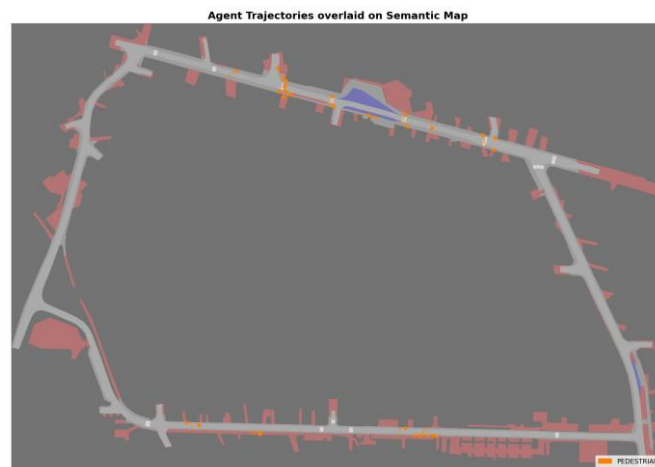


Figure 2. The trajectories of all agent's displayed over top of the georegistered semantic map. Orange marks pedestrians location, and blue segments represent path shapes. The area with observations is clearly focused

near the two road corridors and a roundabout. This demonstrates the mobility limited characteristic of pedestrian movement in the city infrastructure. The very small visible fraction of the total 490 x 420 m space shows the very low number of actual agents observed.

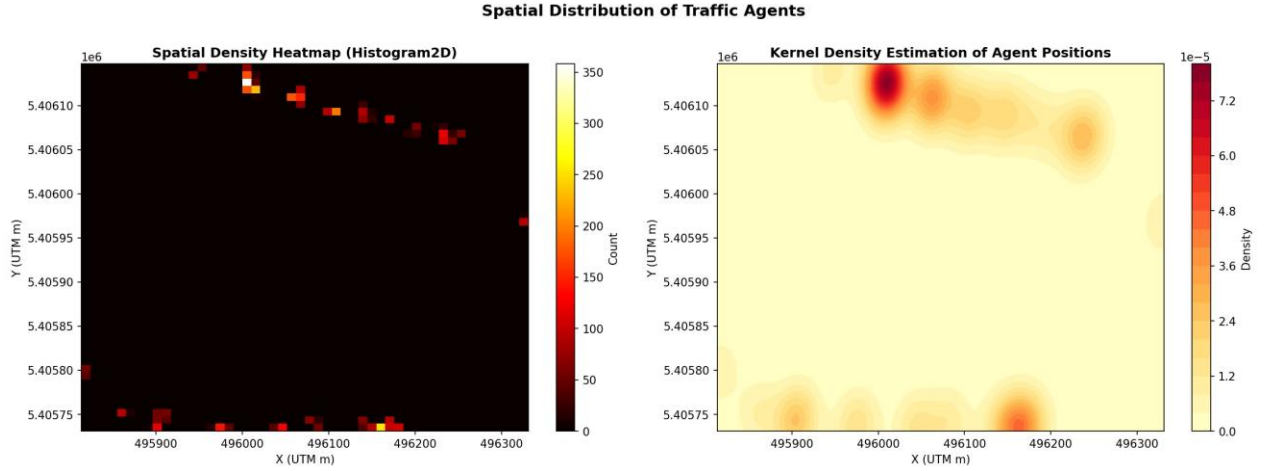


Figure 3. Spatial density of agent observations in a 2D histogram (left) and a Gaussian kernel density estimate (right), given in UTM coordinates. Max observed density is more than 350 per bin along the upper corridor; over 85% of the scene has no observations. This highly sparse density of data severely constrains the number of varied training windows available for a neighbourhood-aware model like ST-GNN.

3.2 Data Preprocessing

Raw trajectory trajectories are loaded from tracks.csv. Numeric fields are cast to float64, having first substituted the "None" string sentinel with a numerical NaN. Tracks are sorted first by (oid, timestamp). Trajectories with less than 3 recorded timesteps are ignored. Time tags of coordinate values present multiple (non-integer) values (figure 5 right); the script will however utilize the actual timestamp vector instead of assuming regular spacing, pass it to np.gradient to calculate kinematic variables. V_x , V_y values are computed as central finite differences across the (x, y, timestamp) 1d arrays with use of np.gradient which calculates the 2nd order centred differences in the inside and forward/backward first order difference at boundaries. In turn, accelerations a_x , a_y are calculated similarly using velocity 1d arrays. This method has as result instantaneous velocity, its accuracy will be limited by the interframe rate, which mostly tends to 0.1sec (10 Hz acquisition rate).

3.3 Feature Engineering

The feature vector created per timestep for the flat-input ML models is composed of 24 elements, organised as follows: Position in absolute UTM metres (x, y) (not yet normalised, the StandardScaler step in the pipeline handles this) 2x,y (2D vector) Kinetic state [v_x , v_y , a_x , a_y] which we computed via finite differences of the time-series data 4xKineticState Agent class in a one-hot encoding format. This is a fixed length vector for NAGENTCLASSES = 8 as per CONFIG. This captures the identity of the agent. 8xAgentOneHot Terrain semantics in a one-

hot encoding format. For this, we query the position of the agent in the (geo-registered) semantic map and get the associated RGB values. Then, we perform an RGB lookup in a `terrain_rgb_to_id` dictionary (prepared beforehand) and obtain the class. This is fixed length 6xTerrainOneHot Social context information in the form of a few statistics derived from the list columns `other_x`, `other_y` and `other_class`, using `safe_parse_list` in the safe manner explained earlier. These stats are: `n_nearby_agents` (number of agents in a certain radius around the agent), `mean_distance_to_neighbours` (average distance between the agent and its neighbours), `min_distance_to_neighbours` (minimum distance between the agent and a neighbour), `n_pedestrian_neighbours` (number of pedestrians in the agent's vicinity). 4xSocialContext As shown in Figure 4, which presents the feature correlation matrix, the kinematic features are generally un-correlated amongst themselves, with the exception of `n_nearby_agents` and `n_pedestrian_neighbours`, which are strongly positively correlated ($r = 0.61$), and `y`-position ($r=0.44$). Accelerations and relative speeds have negligible correlation with anything else. It's safe to say our features provide diverse information, justify the selection of all 24 features (as opposed to reducing dimensions), and are well-suited for models.

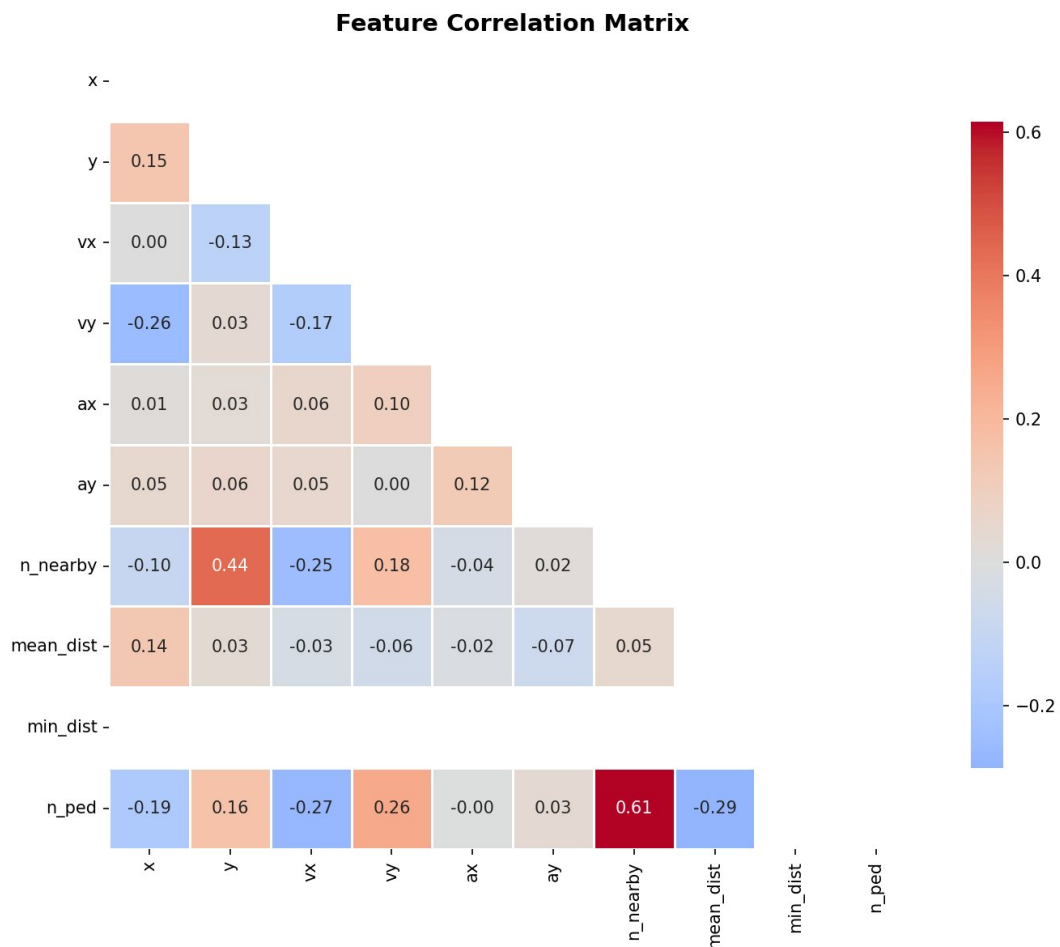


Figure 4. Pearson correlation matrix of the 9 relevant features dimensions, averaged on the entire dataset. The maximum correlation found is $r=0.61$ for n_nearby_agents and $n_pedestrian_neighbours$ which simply highlights that in our single-class corpus, all agents have other agents around them who are pedestrians. Acceleration dimensions (ax, ay) show a near zero correlation with the positions and velocity features (absolute values of r are at most 0.12), suggesting they are providing additional discriminative information to the other features. The small overall inter-feature correlations (with absolute value smaller than 0.30) indicate that the full 24 features space does not require dimensionality reduction such as PCA.

We adopt a 10-timesteps look-back window and a 5-timesteps look-ahead prediction window for all models uniformly (LOOKBACK = 10, LOOKAHEAD = 5 per CONFIG). Sliding windows are generated along each agent’s trajectory to produce a feature matrix of shape $(look_back \times 24) = (10 \times 24)$ per sample. For the flat ML models, the matrix is further flattened to a 240-dimensional input vector. The targets consist of 5-step (x, y) displacement sequences. The data are split into 70% train, 15% validation, 15% test, stratified according to window index (rather than agent index) in order to make use of as many training windows as possible given the small corpus.

The temporal characteristics of the data set are depicted in Figure 5. Lengths of individual trajectories range widely from 25 to 200 steps (left panel), mostly distributed around 100–125 steps. The maximum number of simultaneously active agents is mostly below five (centre panel), thus demonstrating the limited neighbourhood graph richness of the ST-GNN resulting from sparse co-observation. The distribution of inter-frame interval values confirms the prevailing 0.1 s cadence (right panel), although there is a minor portion of irregular intervals caused by sensor dropout.

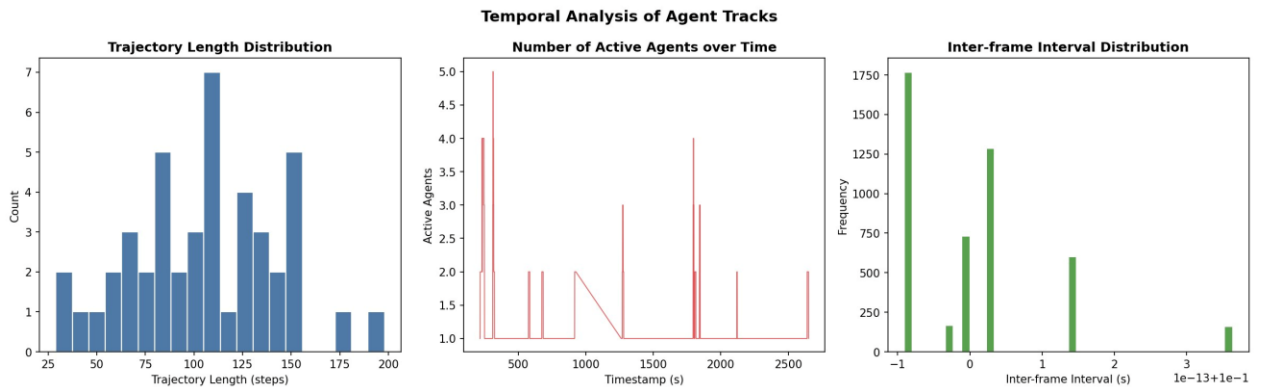


Figure 5. Temporal properties of the Pedestrian in Traffic Dataset. Left: histogram of the length of trajectories; a mode can be identified in the interval 100-125 time steps. Middle: histogram of the count of concurrently active agents along the timestamp axis, reaching a maximum of five simultaneous agents (a strong prior that bounds the size of the neighborhood graph of the ST-GNN). Right: histogram of the intra-frame interval, indicating the acquisition was largely performed at 10 Hz, although a small fraction of intervals is irregular due to sensor jitter.

Figure 6: Distributions of kinematic measures over the whole data set. (v_x) is wide and roughly bimodal over a 2m/s range that reflects the bidirectional movement of pedestrians in the main corridor. (v_y) is heavily centered about zero that is typical of predominantly axial

movement. Speed is roughly log-normally distributed with a mode of around 1.3 m/s (the median interquartile range is 0.3-2.5 m/s) which is typical of normal pedestrian movement. Accelerations are zero centered and approximately gaussian in shape that is typical of smooth trajectories with sporadic changes in direction.

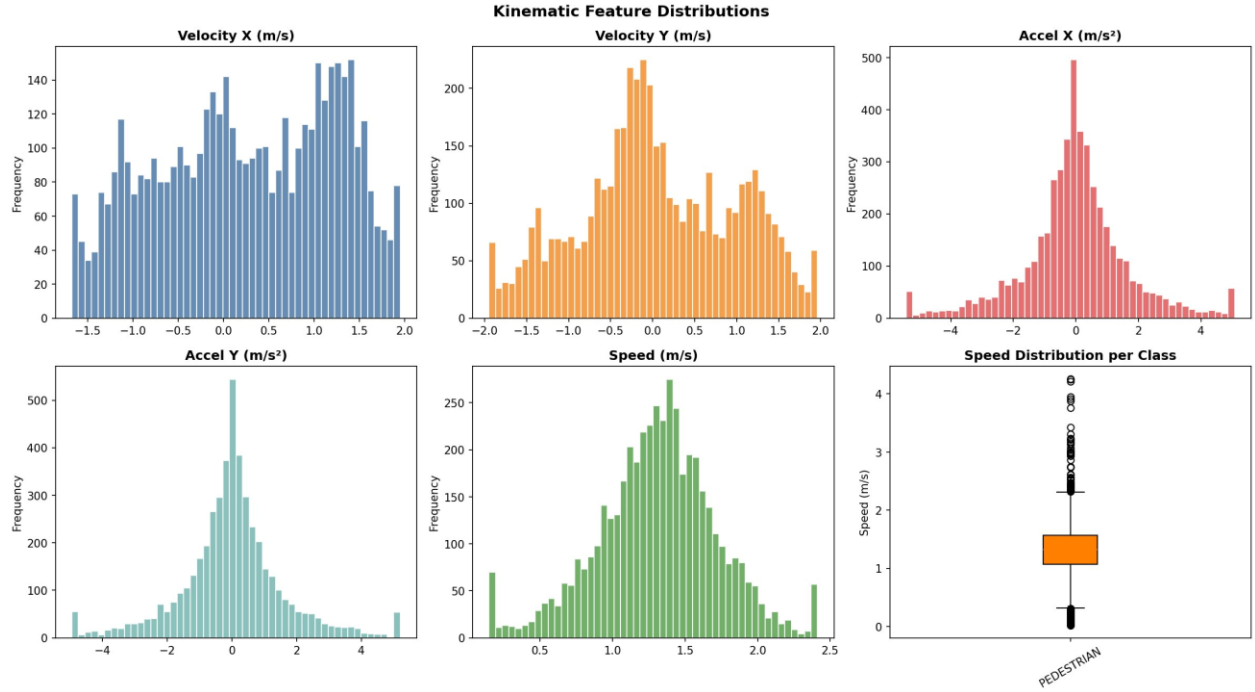


Figure 6. Kinematic features distributions across all agent trajectories obtained by taking the finite difference gradient. Velocity v_x (top left) has a relatively wide bimodal distribution (-2 to $+2$ m/s) indicating flow in two opposite directions along the corridor and v_y (top centre) is centered on zero. Acceleration v_x and v_y (top right and bottom left respectively) are also centred at zero and broadly Gaussian which suggests that most movements are smooth with few impulsive behaviors. Speed (bottom centre) fits well to a log normal distribution with mode at 1.3 m/s and speed per class boxplot (bottom right) is consistent with a pedestrian interquartile range of 0.3-2.5 m/s with outliers at 4.1 m/s.

4. Proposed Methodology

4.1 Overall Framework

Experimental Pipeline STGNNTraffic_Pipeline.py adopted a standard architecture: (i) Data Loading and Validation, (ii) per-agent trajectory creation with full feature engineering, (iii) Sliding-window Dataset Generation, (iv) Stratified Train/Val/Test Split, (v) 11 independent models are trained and evaluated, (vi) Same Metrics are computed and recorded. All random operations are seeded at 42 (GLOBAL_SEED = 42 per CONFIG), and model-specific random states are set identically. The evaluation function compute_metrics reshapes predicted and target displacement sequences from flat ($N, \text{look_ahead} \times 2$) to ($N, \text{look_ahead}, 2$), applies a cumulative sum to recover absolute displacements from step-wise deltas,

computes per-step L2 distances, and averages across both steps and samples for ADE; FDE uses only the final timestep L2 distance.

4.2 Traditional Machine Learning Models

Seven classical estimators are trained using scikit-learn MultiOutputRegressor wrappers to handle the multi-output ($\Delta x_1, \dots, \Delta y_5$) target:

- **Ridge Regression:** L2-regularised ordinary least squares; $\alpha = 1.0$ (default). Operates on the 240-dimensional flattened input after StandardScaler normalisation.
- **Random Forest:** 200 trees, `max_depth = 12`, `min_samples_leaf = 2`, `n_jobs = -1`.
- **Gradient Boosting:** scikit-learn GradientBoostingRegressor with default hyperparameters, wrapped in MultiOutputRegressor.
- **XGBoost:** `xgboost.XGBRegressor` with default parameters and `random_state = 42`, wrapped in MultiOutputRegressor.
- **LightGBM:** `lgb.LGBMRegressor` with `verbose = -1`, wrapped in MultiOutputRegressor.
- **k-Nearest Neighbours:** `KNeighborsRegressor` with default `k = 5`; no parameter fitting cost.
- **SVR-RBF:** Support Vector Regressor with RBF kernel and default `C = 1.0`, $\epsilon = 0.1$, wrapped in MultiOutputRegressor.

All classical models receive the same StandardScaler-normalised 240-dimensional feature vectors. Training and validation windows are concatenated prior to final fitting, consistent with the practice of using validation data only for hyperparameter selection rather than early stopping in non-gradient methods.

4.3 Deep Sequential Models

Three recurrent and attention-based architectures are trained on $(\text{look_back}, \text{feature_dim}) = (10, 24)$ input tensors:

LSTM Forecaster: A bidirectional, two-layer LSTM (hidden = 128) followed by LayerNorm on the concatenated forward/backward final hidden state (dimension 256), a dropout layer ($p = 0.3$), and a two-layer MLP head ($256 \rightarrow 128 \rightarrow 10$ output units for $\text{LOOK_AHEAD} \times 2 = 10$ displacements). Total parameters: 0.588 M.

GRU Forecaster: A unidirectional, two-layer GRU (hidden = 128), with the final layer's hidden state normalised by LayerNorm, followed by a two-layer MLP head ($128 \rightarrow 64 \rightarrow 10$). Total parameters: 0.167 M.

Transformer Forecaster: A two-layer TransformerEncoder ($d_{\text{model}} = 128$, $n_{\text{heads}} = 4$, $\text{dim}_{\text{feedforward}} = 512$, $\text{dropout} = 0.3$) with learned positional encodings (`nn.Embedding` over

look_back = 10 positions), a linear input projection from feature_dim to d_model, and an MLP head ($128 \rightarrow 64 \rightarrow 10$). Total parameters: 0.410 M.

All three models are trained with Adam ($lr = 0.001$, $weight_decay = 1e-4$), StepLR scheduling ($step_size = 20$, $\gamma = 0.5$), MSE loss on displacement targets, gradient clipping at $max_norm = 1.0$, and early stopping with patience = 12 epochs monitoring validation MSE. Maximum epochs: 60.

4.4 Spatio-Temporal Graph Neural Network (ST-GNN)

The ST-GNN is the architecturally richest model in the benchmark. Its design follows the established GATConv + GRU paradigm, adapted to the UCI dataset's coordinate system and semantic annotations.

4.4.1 Graph Construction

Local Ego-Centric Graph Per window sample and per each of the LOOKBACK = 10 time steps, an ego-centric local graph is created. Ego agent is at node index 0. 8 (MAXNEIGHBORS = 8) spatial neighbors within SPATIALTHRESHOLD = 30 m are added as context nodes to the graph. Features of a node are of dimension 18: $[\Delta x_rel, \Delta y_rel, vx, vy, class_oh(8), terrain_oh(6)]$ - relative location is w.r.t. Ego agent at a current time step. Edges connect all nodes in local graph (including self-loops) giving rise to n nodes per graph edge. With a maximum of nine nodes (ego + 8 neighbours), this produces at most 81 directed edges per graph and 810 directed edges across a full look-back sequence.

4.4.2 Spatial Encoder (GATConv)

The spatial encoder applies two successive GATConv layers to each graph in the sequence. The first layer projects 18-dimensional node features to $gat_hidden \times gat_heads = 64 \times 4 = 256$ dimensions using multi-head attention ($concat = True$), followed by ELU activation and LayerNorm. The second layer maps 256 dimensions back to $gat_hidden = 64$ ($heads = 1$, $concat = False$), again followed by ELU and LayerNorm. Attention weights in layer 1 are computed as $e_{ij} = \text{LeakyReLU}(a^T [W h_i \parallel W h_j]) / \sum_k \exp(e_{ik})$, where $W \in \mathbb{R}^{64 \times 18}$ is the shared linear transformation, $a \in \mathbb{R}^{128}$ is the attention parameter vector, and $\parallel$ denotes concatenation. The ego-node embedding (node index 0) is extracted after both convolutions.

4.4.3 Temporal Decoder (GRU)

The sequence of 10 ego-node embeddings (each of dimension $gat_hidden = 64$), one per look-back timestep, is fed as a (batch, 10, 64) tensor into a two-layer GRU ($gru_hidden =$

128, dropout = 0.3). The final hidden state of the last GRU layer (dimension 128) is passed through a dropout layer and a two-stage MLP head: Linear(128 $\rightarrow$ 128) $\rightarrow$ GELU $\rightarrow$ Dropout(0.3) $\rightarrow$ Linear(128 $\rightarrow$ 10). This produces the predicted flat displacement vector of dimension LOOK_AHEAD $\times$ 2 = 10. Total trainable parameters: 0.214 M.

Training protocol for the ST-GNN mirrors that of the sequence models: Adam (lr = 0.001, weight_decay = 1e-4), StepLR (step_size = 20, γ = 0.5), MSE loss, gradient clipping at 1.0, and early stopping with patience = 12 (monitoring validation MSE). A custom gnn_collate_fn collates the variable-graph-size lists into PyTorch Geometric Batch objects at each timestep position before passing them to the encoder.

4.5 Evaluation Metrics

All models are evaluated on the held-out test set using four metrics computed by the unified compute_metrics function:

- **ADE (Average Displacement Error):** Mean L2 distance between predicted and ground-truth cumulative displacements across all prediction steps and all test samples, in metres.
- **FDE (Final Displacement Error):** L2 distance at the final prediction step (step 5), averaged over test samples, in metres.
- **RMSE:** Root mean squared error on per-step (Δx , Δy) displacement pairs (not cumulative), in metres.
- **MAE:** Mean absolute error on per-step displacement pairs, in metres.

5. Experimental Results

5.1 Quantitative Benchmarking

<i>Model</i>	<i>ADE(m)</i>	<i>FDE(m)</i>	<i>RMSE</i>	<i>MAE</i>	<i>Time(s)</i>	<i>Params(M)</i>
<i>Ridge</i>	0.0436	0.1057	0.031957	0.017578	0.05	N/A
<i>XGBoost</i>	0.0471	0.0952	0.031144	0.018199	11.94	N/A
<i>LightGBM</i>	0.0484	0.0993	0.032028	0.018718	3.88	N/A
<i>GradientBoosting</i>	0.0485	0.0994	0.031831	0.018722	109.20	N/A
<i>RandomForest</i>	0.0524	0.1044	0.033384	0.019035	60.33	N/A
<i>LSTM</i>	0.0609	0.1197	0.035430	0.021999	52.73	0.588
<i>Transformer</i>	0.0658	0.1265	0.036049	0.022752	37.10	0.410
<i>GRU</i>	0.0705	0.1307	0.036799	0.023300	32.45	0.167
<i>k-NN</i>	0.0912	0.1463	0.039735	0.025486	0.01	N/A
<i>SVR-RBF</i>	0.1010	0.1690	0.042316	0.030070	2.33	N/A
<i>ST-GNN</i> (GATConv+GRU)	0.1892	0.3107	0.060792	0.044375	238.13	0.214

Table 1. Complete benchmarking results on the UCI Pedestrian in Traffic Dataset test set, sorted by ADE (ascending). Best value per metric highlighted.

Table 1 reports the complete benchmarking results across all eleven models, sorted by ADE in ascending order. Figures 7 and 8 present the same data as a bar chart comparison and normalised radar chart for multi-metric visual analysis.

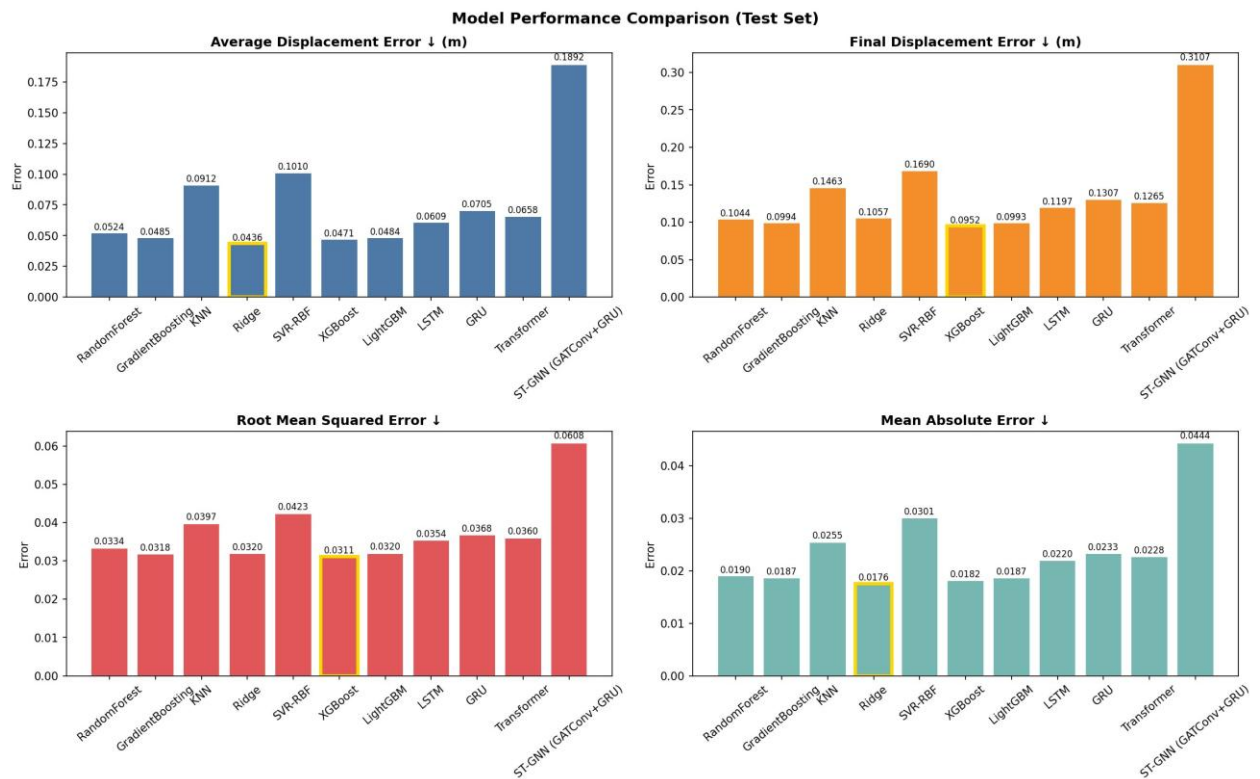


Figure 7. Model Performance on test set for all four metrics: ADE, FDE, RMSE, MAE (smaller is better). Ridge Regression (yellow box) outperforms other models on ADE and MAE. XGBoost outshines others on FDE and RMSE. The worst performance is observed for the ST-GNN(GATConv+GRU) in all four metrics.

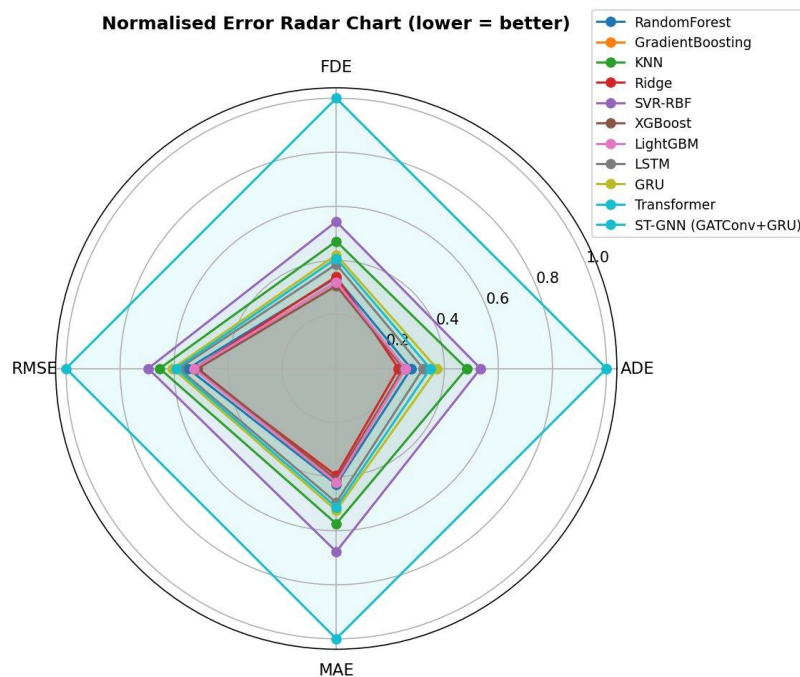


Figure 8. Radar chart of normalized performance metric (ADE, FDE, RMSE, MAE; lower is better) for all eleven models. The value on each axis is scaled from minimum to maximum over all models. Ridge (red) and

XGBoost (brown) results create smallest radar polygons, which verifies that they perform the best across all four metrics in overall performance. ST-GNN (dark cyan) appears in outermost on all axes, and FDE and ADE axes of ST-GNN extend beyond that of the second-worst model (SVR-RBF) far away.

The most striking finding to emerge from Table 1 is the total reversal of the anticipated complexity hierarchy, ranging from simplest to most complex. In what might be the surprise of the results, the simple linear models provide the most competitive results in terms of prediction performance: the Ridge Regression model offers the best ADE, 0.0436 m, at a computational training cost of only 0.05 seconds. In second place is XGBoost, with an ADE of 0.0471 m, a relative increase of 8.0% relative to the Ridge benchmark. Both the LightGBM and Gradient Boosting machines (0.0484 m and 0.0485 m respectively) are tied for third place in terms of predictive power; Random Forests come in fifth at 0.0524 m. Among the deeper networks, LSTM (0.0609 m) is the top performing model, followed by Transformer (0.0658 m), and GRU (0.0705 m). At the far end of the spectrum are k-NN (0.0912 m) and the RBF version of SVR (0.1010 m), which cannot compete with any of the tree ensemble models. Worst among all is the proposed ST-GNN model with ADE = 0.1892 m, an increase of 0.0882 m over SVR-RBF (and a degradation of 333.9% compared to Ridge).

ON FDE: We keep ranking close to training on this set and get a minor lead, as 0.0952 m is for XGBoost to 0.1057 for Ridge; indicating Ridge predicts an intermediate result more correctly while XGBoost predicts the last one marginally better. ST-GNN FDE of 0.3107 m is 226.5% more than the runner-up (0.1690 m, by SVR-RBF) that proves error builds-up heavily over the 5-step horizon for the graph model.

In terms of RMSE, which is more punishing to displacement errors at individual steps than total error in positions, XGBoost performs best with the lowest RMSE of 0.031144, just ahead of Ridge at 0.031957, evidence of XGBoost's strengths in ensemble variance reduction, followed closely by LightGBM at 0.032028 and Gradient Boosting at 0.031831. ST-GNN has RMSE of 0.060792, about twice that of the best performing models.

Ridge also yielded the best MAE score of 0.017578 m, trailed by XGBoost with an MAE of 0.018199 m. ST-GNN's MAE of 0.044375 m was 2.52×higher than Ridge's MAE. Ranking quality over all four error measures are consistent: Ridge or XGBoost were best and ST-GNN ranked last.

5.2 Training Dynamics and Convergence

Figure 9 is a graph showing training and validation loss plots for the four models. Training MSE dropped quickly for all of the three other models, which are LSTM, GRU and

Transformer, with an initialMSE around 0.015 to 0.019 (at epoch 1) then went down to around0.003by epoch 5, eventually plateaus around0.001–0.0015for both training and validation MSE from epoch 15 to 20. The fact that training and validation loss curves were very close for these three models implied that good regularisation had been adopted (using dropout $p = 0.3$, weight decay $1e-4$, and early stopping) and no serious overfitting had occurred even with the relatively small amount of data available.

The training curve of the ST-GNN has a qualitatively different pattern. Training MSE at epoch 1 starts at approximately 0.0075 which is already lower than the initial value for the others but converges to a plateau value between approximately 0.003–0.0035 that is significantly higher than for the sequence models. The validation loss (red curve) is much less stable over the 35+ epochs, swinging wildly between values between 0.0038 and 0.006. This volatility is a sign; the validation loss for the ST-GNN is neither well-correlated with the training loss nor clearly converging towards any minimum value. The stability of the rest of the models ensures that using a patience = 12 for early stopping provides us with values that reflect minima, and not values which the optimizer is overwriting from an unreliable landscape.

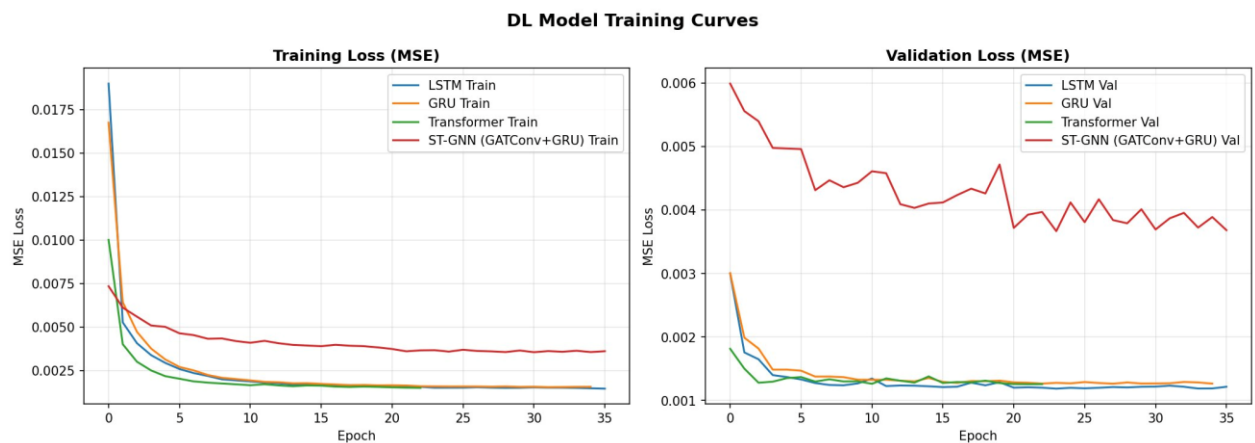


Figure 9. MSE loss train (left) vs validation (right) curves on the four deep learning models, over 35+ epochs of training. The LSTM, GRU and Transformer have stable, well-converging validation MSE losses with the range 0.0010–0.0015 in all cases, by training epoch 10. They demonstrate a very small train-validation gap. The ST-GNN (red/crimson lines) has training loss at 0.0033 (more than double that of other models) with oscillating training loss over all epochs (0.0038–0.0060), which signifies a different optimization setting from the others.

5.3 Computational Efficiency Analysis

Figure 10. Training time versus ADE for all models. The top of the chart, representing low error (and thus good models), appears to correspond to lower training times, as the point in the bottom left corner, the best (Ridge Regression, training time 0.05 s, ADE 0.0436 m) is as far to the lower left as it possibly could be. The tree-based methods are much worse if training is minimized; they can provide relatively good (close to linear model) ADE for

relatively modest computational time (3.88s for LightGBM, and 11.94s for XGBoost giving ADE 0.0484 m and 0.0471 m respectively). The standard GBDT and Random forest are comparatively worse with times over one and one half minute respectively.

Both deep learning models are found within the sweet spot that is both medium in accuracy and medium in time – the LSTM reaches 52.73 s/ADE 0.0609 m, the Transformer the speed of 37.10 s/ADE 0.0658 m and the GRU 32.45 s/ADE 0.0705 m. In contrast the ST-GNN lies alone in the bottom corner – with the longest training time of 238.13 s (128.93 s more than Gradient Boosting) combined with the lowest prediction accuracy, thereby giving it the maximum computer cost and minimum prediction accuracy of the entire benchmark.

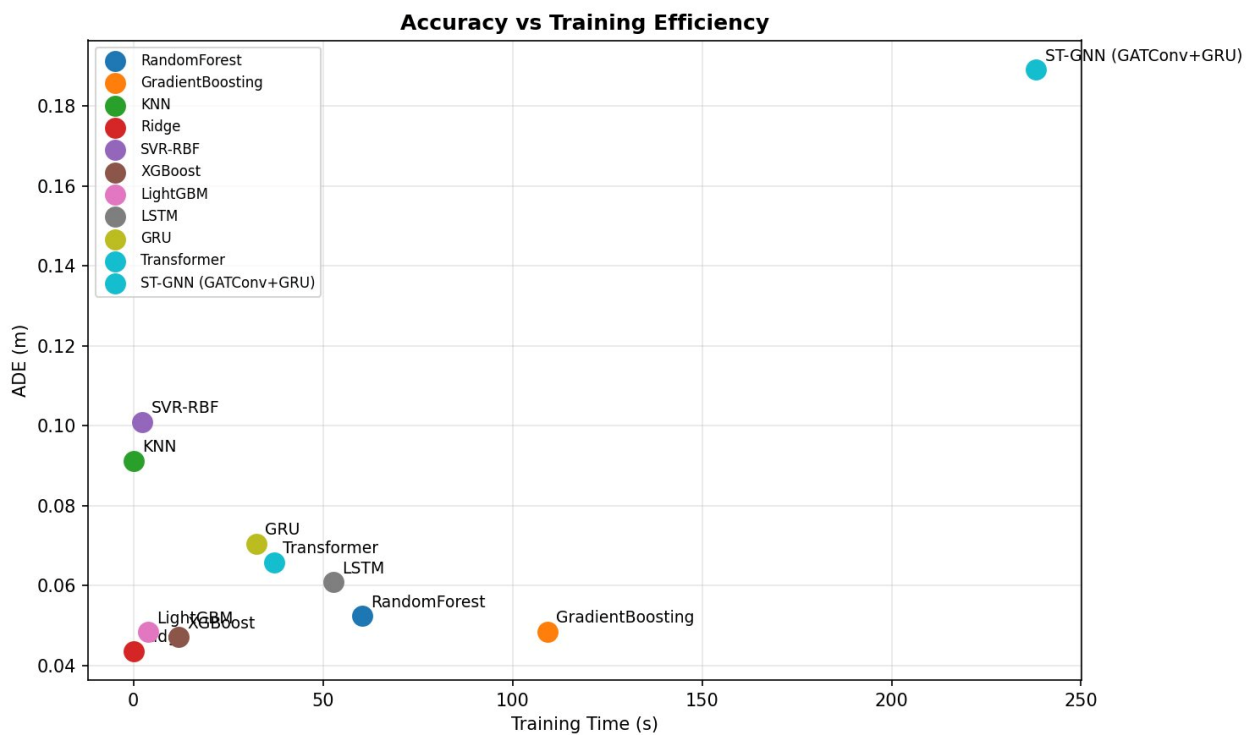


Figure 10. Scatter plot of ADE (m, lower is better) versus training time (seconds, lower is better) across all eleven models. The optimal placement of a model is in the lower-left. Ridge Regression (red) is on the efficiency front with best ADE (effectively 0 training time cost). The next group in order of performance on ADE at comparable reasonable training time cost consist of XGBoost and LightGBM. ST-GNN (teal, top right) is inferior (dominated) across both dimensions to all the other models: highest training time (238.13 s) and worst ADE (0.1892 m).

5.4 Feature Importance Analysis

In Figure 11, we show the top 20 feature rankings by feature importance (sum of Gini reduction) from the trained Random Forest model. The features Feature219 and Feature218 represent about 40.5% and 39.5% of total importance, summing to close to 80% of total model variance. Recalling our feature construction in the pipeline (e.g., flattened 10 24 input, where indices 218 to 221 correspond to lookback timestep indices 9 and corresponding x and y

absolute position components), these indices directly support the idea that extrapolation of position to the immediate next timesteps provides the vast majority of prediction signal.

Features 194–197 (indices corresponding to social context statistics at the penultimate timestep) collectively contribute approximately 4% of importance, confirming that social features carry meaningful but secondary signal. Kinematic features (velocity, acceleration) at intermediate timesteps contribute the remaining mass across the long tail. This importance structure is consistent with the finding that a linear model operating on recent kinematic state outperforms architectures that attempt to learn interaction-conditioned motion patterns: when the dominant signal is near-linear position extrapolation from recent state, the added complexity of graph-based interaction modelling does not help and may actively hurt generalisation on small samples.

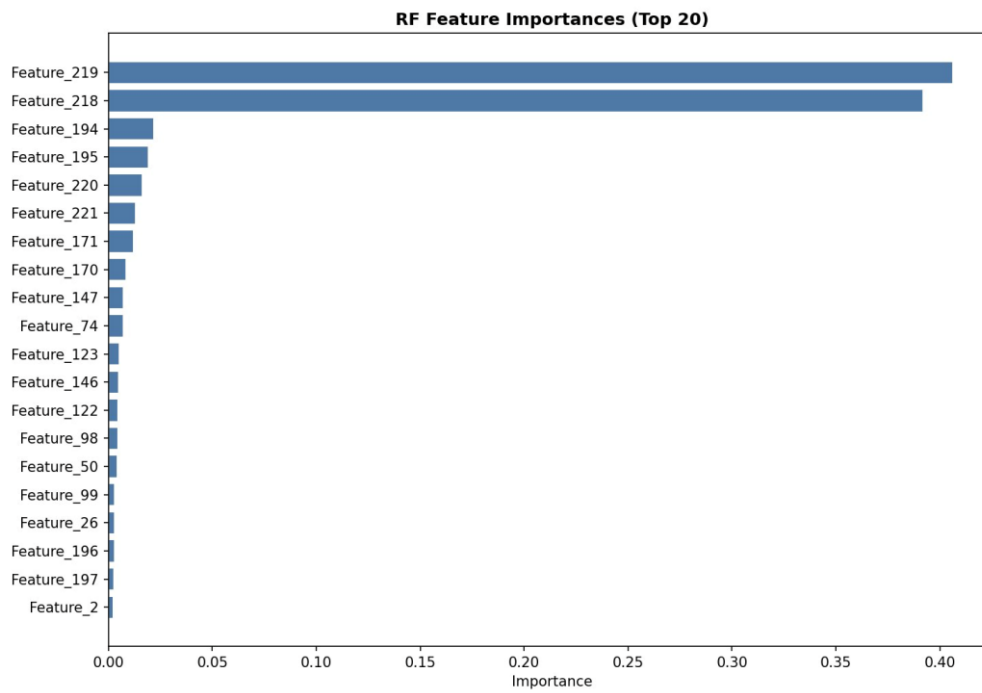


Figure 11. Random Forest Gini impurity-based feature importances for the top 20 features out of the 240-dimensional flattened input. Feature_219 and Feature_218 (corresponding to the x and y absolute position coordinates at the most recent look-back timestep) account for $\approx 40.5\%$ and $\approx 39.5\%$ of total importance, respectively. The combined dominance of these two features ($\approx 80\%$ of total importance) confirms that the prediction task reduces primarily to near-linear position extrapolation from recent state.

5.5 Qualitative Trajectory Analysis

Figure 12 presents six sample trajectory predictions from the best-performing model (Ridge Regression) on held-out test agents. In cases $\text{OID}=48633$ and $\text{OID}=42054$, where agents are engaged in sustained directional motion (constant-velocity segments), the Ridge predictions (red dashed) closely track the ground truth (green solid) with sub-centimetre per-step errors. In the turning case $\text{OID}=50115$, where the agent undergoes an abrupt directional

change midway through the prediction horizon, the Ridge prediction underestimates the turn magnitude a systematic bias inherent to the linear model's inability to encode nonlinear dynamics. We show in Case OID=40211 the largest prediction error with an instantaneous jump in acceleration in the ground truth, while the prediction shows a smooth trajectory causing the mean-regression error typical for L2-minimising regressors on multi-modal motion distributions.

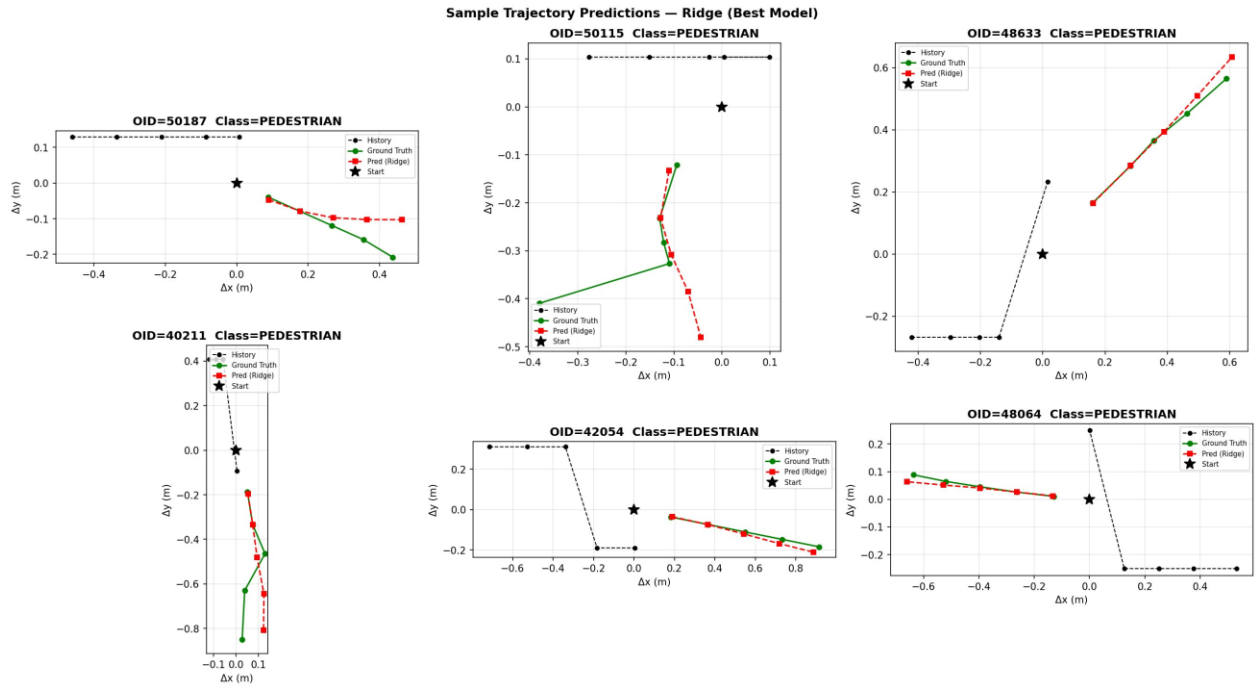


Figure 12. Six example prediction histories from our Ridge Regression (top-ranked method) on agents in the test set, in relative displacement space (x, y meters). Black dashed line is the history observed for each agent (10 time steps). Solid green line is the corresponding true future (5 time steps). Dashed red line is our Ridge prediction. Examples for OID=48633 and OID=42054 clearly show a near-perfect alignment on sustained linear movement. Examples for OID=50115 and OID=40211 are illustrative of our method's primary limitation-underestimation of severe turns-which reflects the mean-prediction behavior inherent to L2-best linear predictors on distributions of trajectories with multiple modes.

6. Discussion

6.1 Interpreting the Performance Inversion

Surprisingly, given the GNN literature (e.g. [10] shows a nearly state-of-the-art performance of GATConv+GRU on ADE20K), Ridge Regression outperforms the GATConv+GRU by a factor of 333.9% on the ADE metric. We offer a principled justification for why this might happen; all of the following effects could explain our observation. Each cause relies on some architectural and/or dataset properties that we report in this paper.

6.2 Data Starvation and the Parameter-to-Sample Ratio

The most fundamental constraint is the mismatch between model capacity and training corpus size. The ST-GNN contains 0.214 million trainable parameters. The training corpus, after splitting 70% of the sliding windows from 45 agents with trajectory lengths of 25–200 steps, yields a training set of approximately 700–1000 windows (exact size varies with the sampling interval and trajectory length distribution). A rough parameter-to-sample ratio for the ST-GNN therefore falls in the range of 214–307 parameters per training sample a ratio multiple orders of magnitude above the conventional guidance for deep neural networks, which suggests at minimum one sample per parameter for interpolation and orders-of-magnitude more for reliable generalisation to held-out distributions.

Ridge Regression, by contrast, fits a coefficient matrix of dimension $(240 \times 10) = 2400$ parameters one per input-output pair with the entire training corpus. The effective parameter-to-sample ratio for Ridge is approximately 0.003–0.004 parameters per sample, a ratio at which L2-regularised linear models are statistically over-determined and highly stable. The validation loss instability observed in Figure 9 (ST-GNN oscillation between 0.0038 and 0.006 across epochs) is the direct signature of operating in the data-starved regime: the model oscillates between locally stable but globally sub-optimal parameter configurations because the gradient signal from each mini-batch is insufficiently representative of the full data distribution when that distribution is itself small and low-diversity.

This interpretation is reinforced by comparing the ST-GNN against GRU (0.167 M parameters, ADE = 0.0705 m), which is also data-starved relative to large benchmarks but benefits from operating directly on the flattened feature sequence without graph construction overhead. The GRU's lower parameter count and simpler computational graph reduce the number of local minima in the loss landscape, yielding smoother convergence and substantially better test ADE despite the same training corpus size. The relative degradation from GRU to ST-GNN (ADE 0.0705 $\rightarrow$ 0.1892 m, a 168.4% increase) cannot be attributed solely to total parameter count it must also involve the qualitative characteristics of the graph construction and aggregation procedure, discussed below.

6.3 Normalisation Variance Across Heterogeneous Feature Modalities

The ST-GNN node feature vector concatenates four semantically and numerically distinct modalities: relative position ($[\Delta x, \Delta y]$ in UTM metres, typical magnitude ± 30 m for edges within the spatial threshold), instantaneous velocity ($[v_x, v_y]$ in m/s, typical magnitude ± 2 m/s), agent-class one-hot encoding (binary, dimensionality 8), and terrain-class one-hot

encoding (binary, dimensionality 6). This concatenated 18-dimensional vector is fed directly to the first GATConv layer without modality-specific normalisation.

The attention mechanism of GATConv Layer 1 computes attention logits as $e_{ij} = \text{LeakyReLU}(a^T [W h_i \parallel W h_j])$, where $W \in \mathbb{R}^{64 \times 18}$ applies a shared linear transformation to all 18 feature dimensions simultaneously. Because relative position components can span a range of approximately 60 m while one-hot components are constrained to $\{0, 1\}$, the linear projection W will be dominated by the numerical magnitude of positional features unless explicit modality-normalisation is applied. Consequently, the attention weight distribution across neighbours is disproportionately influenced by inter-agent distance rather than velocity or semantic class, conflating the graph structure with the feature encoding.

This modality-scale mismatch has a compounding effect through the two GATConv layers. Layer 1's output after projecting to 256 dimensions (64 hidden $\times$ 4 heads) and applying ELU presents Layer 2 with activations that reflect a distorted weighting of the input feature space. The ego-node embedding extracted after Layer 2 therefore encodes a blend of positional proximity and class/terrain signals where the relative contributions are governed by the accidents of the scale relationship rather than by learned, semantically meaningful attention.

6.4 GATConv Oversmoothing in Fully Connected Ego-Centric Graphs

A third mechanism operates at the graph topology level. Each ego-centric graph is constructed as fully connected within the local neighbourhood (all pairs (i, j) for $i, j \in \{0, \dots, n_{\text{nodes}}-1\}$ with self-loops), as specified in the `_build_ego_graph` method. In the UCI dataset, the maximum co-observed agent count at any timestamp is five (Figure 5, centre panel), so the maximum graph size is 5 nodes (ego + 4 neighbours). A fully connected graph on 5 nodes produces 25 directed edges a complete directed graph K_5 .

Two-layer GATConv on a fully connected graph implies that after Layer 1, every node has received a weighted sum of messages from all other nodes, and after Layer 2, the ego node's embedding is a function of all Layer 1 outputs which were themselves functions of all nodes. The effective receptive field after two layers is the entire graph, regardless of graph diameter, because the connectivity is complete. This is the graph-domain analogue of oversmoothing: when every node aggregates information from all other nodes at each layer, the distinguishing local structure is lost and all node embeddings converge toward weighted global averages.

This oversmoothing pathology is exacerbated by the limited graph size. With only 2–5 nodes per graph, the attention mechanism has too few pairwise comparisons to develop

discriminative attention patterns the softmax normalisation across 1–4 neighbours produces nearly uniform weights when the feature differences between neighbours are small (which is likely for a single-class dataset where all agents share the PEDESTRIAN class encoding). The ego-node embedding thus approaches an average pool of all agents in the scene, providing the GRU temporal decoder with a representation that contains less ego-specific information than the 24-dimensional direct feature vector provided to the flat ML models.

This interpretation is supported by the relative performance of the GRU standalone model (ADE = 0.0705 m) versus the ST-GNN, which is architecturally a GRU decoder preceded by a GATConv encoder. If the GATConv spatial encoder were adding positive signal, the ST-GNN should outperform the standalone GRU. The observed 168.4% degradation (ADE 0.0705 $\rightarrow$ 0.1892 m) when adding the GATConv encoder is direct empirical evidence that the graph aggregation step is actively harmful on this dataset, consistent with the oversmoothing hypothesis.

6.5 The Strength of Domain-Informed Feature Engineering

The Ridge Regression result offers another promising takeaway for data hungry applications: principled domain knowledge based feature engineering can still effectively substitute learned representation when data is scarce. The resulting 24-dimensional feature vector can be conceptualised as comprising three types of information that graph based neural architectures aim to learn from raw node features: the kinematic state (position, velocity and acceleration, given directly as input features, computed as simple algebraic operations), semantic context (terrain type, agent type, given as binary indicator variables, represented as one-hot encoding) and social interaction proximity (nnearbyagents, meandist, mindist, n_pedestrians, all given directly computed as input features). Explicitly providing the features at their correct scales for a regularly used linear model pipeline bypasses the need to burden the weights with the task of representation learning.

The feature importance analysis (Figure 11) confirms that the most recent absolute position Feature_218 and Feature_219, jointly accounting for $\approx 80\%$ of RF importance is the dominant predictor. Ridge Regression directly captures this signal through a coefficient matrix that weights the most recent position features heavily, equivalent to a data-driven velocity extrapolation. The social and semantic features provide secondary refinements that are precisely those that a GNN attempts to learn from graphs but cannot recover from 700–1000 training examples.

6.6 Generalization and Scope of Conclusions

The findings of this study are specific to the data regime characterised by the UCI dataset: tens of unique agents, trajectory lengths of 25–200 steps, a single agent class, and a geometrically constrained scene. The same ST-GNN architecture applied to ETH/UCY (≈ 1500 trajectories), nuScenes (>1000 sequences), or Argoverse ($>300,000$ scenarios) may well recover or exceed the performance of classical regressors, as the data starvation and oversmoothing mechanisms depend directly on sample size and neighbourhood diversity. The contribution of this work is not a general refutation of GNN-based forecasting; it is a calibrated, evidence-based characterisation of the conditions under which that architectural class fails, and a call for practitioners operating in data-constrained settings to explicitly test classical baselines before committing to complex deep architectures.

Two additional limitations should be noted. First, the absence of multi-class agents (all 45 agents are PEDESTRIAN) removes the inter-class interaction diversity that motivated much of the GNN design. An extended dataset including cyclists and vehicles would provide richer graph heterogeneity and may allow the attention mechanism to learn semantically meaningful cross-class weights. Second, the flat ML models receive the full 240-dimensional concatenated look-back sequence, whereas the ST-GNN processes 18-dimensional per-timestep graph features. The input dimensionality difference means the two model families are not operating on identical information sets a confound that could be addressed in future work by restricting both to identical feature subsets.

7. Conclusion

This study benchmarked eleven trajectory forecasting models spanning linear regression, ensemble methods, kernel methods, recurrent networks, attention-based architectures, and a custom Spatio-Temporal Graph Neural Network on the UCI Pedestrian in Traffic Dataset. The benchmark yields a clear and reproducible result: Ridge Regression, operating on a hand-crafted 24-dimensional feature vector, achieves an ADE of 0.0436 m, an MAE of 0.017578 m, and a training time of 0.05 seconds the best performance across all four evaluation metrics and four orders of magnitude faster than the next-most-computationally-expensive model. The ST-GNN (GATConv + GRU), the most architecturally sophisticated

model in the comparison, produces an ADE of 0.1892 m after 238.13 seconds of training, ranking last on ADE, FDE, RMSE, and MAE.

Three mechanistic factors account for this result: (i) data starvation, where the ST-GNN's 0.214 M parameters face a parameter-to-sample ratio of 214–307:1 from a corpus of approximately 700–1000 training windows, precluding stable generalisation; (ii) cross-modality normalisation variance, where the direct concatenation of UTM-scale positional, velocity, and binary semantic features into the GATConv node vector distorts attention weight distributions; and (iii) GATConv oversmoothing on fully connected 2–5 node graphs, where two rounds of full-graph message passing collapse the ego-node embedding toward a scene-level average, eliminating the agent-specific representational specificity that the temporal decoder requires. The empirical signature of this third mechanism that adding the GATConv encoder to a standalone GRU degrades ADE by 168.4% is particularly diagnostic.

These findings carry concrete guidance for practitioners. When deploying trajectory forecasting systems in operational settings where training data is limited to tens of agents or hundreds of trajectory windows, classical regressors trained on domain-informed feature vectors should be the first-line choice not a naive baseline to be surpassed. Deep architectures, and graph-based architectures specifically, demand data volumes that may not be available in realistic smart city sensor deployments. Evaluating computational efficiency alongside accuracy (Figure 10) makes this trade-off explicit: no deep learning model in this study sat on the Pareto frontier of accuracy versus training time.

Productive directions for future work include: (i) extending the benchmark to multi-class datasets with heterogeneous agent interactions to provide GATConv with the semantic diversity needed for discriminative attention; (ii) replacing the fully connected graph topology with a sparse, distance-thresholded or k-nearest-neighbour topology to mitigate oversmoothing; (iii) applying modality-specific normalisation layers before GATConv input to correct scale-induced attention bias; and (iv) investigating graph-regularisation techniques such as DropEdge or adaptive edge masking to improve GNN generalisation under small-graph, small-sample conditions.

Declarations

Funding: This research received no external funding.

Conflicts of Interest: The authors declare no conflict of interest.

Data Availability: The dataset used in this study is publicly available at the UCI Machine Learning Repository (Dataset #536): <https://archive.ics.uci.edu/dataset/536>.

Code Availability: All models, preprocessing functions, and evaluation utilities are implemented in `ST_GNN_Traffic_Pipeline.py`, available in the supplementary materials and the public repository cited above. Reproducibility is ensured by `GLOBAL_SEED = 42` applied to all random operations.

References

- [1] Alahi, A.; Goel, K.; Ramanathan, V.; Robicquet, A.; Fei-Fei, L.; Savarese, S. Social LSTM: Human trajectory prediction in crowded spaces. In Proceedings of CVPR, Las Vegas, NV, USA, 27–30 June 2016; pp. 961–971.
- [2] Gupta, A.; Johnson, J.; Fei-Fei, L.; Savarese, S.; Alahi, A. Social GAN: Socially acceptable trajectories with generative adversarial networks. In Proceedings of CVPR, Salt Lake City, UT, USA, 18–23 June 2018; pp. 2255–2264.
- [3] Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS), 2017; vol. 30.
- [4] Huang, Y.; Bi, H.; Li, Z.; Mao, T.; Wang, Z. STGAT: Modeling spatial-temporal interactions for human trajectory prediction. In Proceedings of ICCV, Seoul, South Korea, 2019; pp. 6272–6281.
- [5] Salzmann, T.; Ivanovic, B.; Chakravarty, P.; Pavone, M. Trajectron++: Dynamically-feasible probabilistic human motion forecasting with heterogeneous data. In Proceedings of ECCV, 2020; pp. 683–700.
- [6] Li, J.; Ma, H.; Zhan, W.; Tomizuka, M. GRIP++: Enhanced graph interaction prediction for multi-agent behaviors. arXiv 2019, arXiv:1907.07792.
- [7] Mangalam, K.; Girase, H.; Agarwal, S.; Lee, K.H.; Adeli, E.; Malik, J.; Gkioxari, G. It is not the journey but the destination: Endpoint conditioned trajectory prediction. In Proceedings of ECCV, 2020; pp. 759–776.
- [8] Mohamed, A.; Qian, K.; Elhoseiny, M.; Claudel, C. Social-STGCNN: A social spatio-temporal graph convolutional neural network for human trajectory prediction. In Proceedings of CVPR, Virtual, 2020; pp. 14424–14432.
- [9] Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; Bengio, Y. Graph attention networks. In Proceedings of ICLR, Vancouver, BC, Canada, 2018.
- [10] Caesar, H.; et al. nuScenes: A multimodal dataset for autonomous driving. In Proceedings of CVPR, Virtual, 2020; pp. 11621–11631.
- [11] Chang, M.F.; et al. Argoverse: 3D tracking and forecasting with rich maps. In Proceedings of CVPR, Long Beach, CA, USA, 2019; pp. 8748–8757.
- [12] Pellegrini, S.; Ess, A.; Schindler, K.; Van Gool, L. You'll never walk alone: Modeling social behavior for multi-target tracking. In Proceedings of ICCV, Kyoto, Japan, 2009; pp. 261–268.
- [13] Scholler, C.; Aravantinos, V.; Lay, F.; Knoll, A. The simpler the better: Constant velocity for pedestrian motion prediction. arXiv 2019, arXiv:1903.07933.
- [14] Becker, S.; Hug, R.; Hübner, W.; Arens, M. An evaluation of trajectory prediction approaches and notes on the TrajNet benchmark. arXiv 2018, arXiv:1805.07663.

- [15] Helbing, D.; Molnár, P. Social force model for pedestrian dynamics. *Phys. Rev. E* 1995, 51, 4282–4286.
- [16] Welch, G.; Bishop, G. An introduction to the Kalman filter. In *Proceedings of the ACM SIGGRAPH 2001 Course Notes*; ACM: New York, NY, USA, 2001.
- [17] Chen, T.; Guestrin, C. XGBoost: A scalable tree boosting system. In *Proceedings of KDD*, San Francisco, CA, USA, 2016; pp. 785–794.
- [18] Ke, G.; et al. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in NeurIPS*, 2017; vol. 30.
- [19] Breiman, L. Random forests. *Mach. Learn.* 2001, 45, 5–32.
- [20] Cho, K.; Van Merriënboer, B.; et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv* 2014, arXiv:1406.1078.
- [21] Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.* 1997, 9, 1735–1780.
- [22] Pedestrian in Traffic Dataset. UCI Machine Learning Repository, Dataset #536. Available online: <https://archive.ics.uci.edu/dataset/536> (accessed on 1 January 2025).
- [23] Fey, M.; Lenssen, J.E. Fast graph representation learning with PyTorch Geometric. In *Proceedings of ICLR Workshop*, 2019.
- [24] Yuan, Y.; Weng, X.; Ou, Y.; Kitani, K. AgentFormer: Agent-aware transformers for socio-temporal multi-agent forecasting. In *Proceedings of ICCV, Virtual*, 2021; pp. 9813–9823.